

Comprehensive Data Engineering Solution for Healthcare Analytics

Name: Srijan Nirmal Kumar Galla

Overview:

This documentation outlines the architecture and components of the Data Pipeline designed to manage the Healthcare datasets. The system includes a Relational Database and cloud storage and workflow automation.

Objectives:

- 1) Data storage
- 2) Data cleaning and transformation
- 3) Workflow automation
- 4) Data analysis
- 5) Data visualization

Data Storage:

Uploaded the gathered datasets in AWS S3 for cloud storage
S3 url: s3://bunny970/Datasets/

Amazon S3 > Buckets > bunny970 > Datasets/

Datasets/

Copy S3 URI

Objects

Properties

Objects (11) Info

Refresh

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
☐	annual-healthcare-expenditure-per-capita.csv	csv	January 12, 2024, 11:39:40 (UTC+05:30)	99.4 KB	Standard
☐	expenditure-of-the-national-health-service-nhs-in-the-uk.csv	csv	January 12, 2024, 11:39:42 (UTC+05:30)	859.0 B	Standard
☐	health-insurance-coverage-in-the-us.csv	csv	January 12, 2024, 11:39:43 (UTC+05:30)	1.3 KB	Standard
☐	health-protection-coverage.csv	csv	January 12, 2024, 11:39:44 (UTC+05:30)	3.6 KB	Standard
☐	percentage-of-persons-without-health-insurance-coverage-us.csv	csv	January 12, 2024, 11:39:45 (UTC+05:30)	1.1 KB	Standard
☐	public-health-expenditure-	csv	January 12, 2024, 11:39:49	53.5 KB	Standard

Designed a relational database to efficiently store and organize the collected data.

Database Schema:

Database name: healthcare_db

Table: exp

Columns:

Entity	nvarchar(50)
Code	nvarchar(50)
Year	smallint
Indicator_Current_health_expenditure_CHE_per_capita_in_PPP_int	float

Table: health_gdp

Columns:

Entity	nvarchar(50)
Code	nvarchar(50)
Year	smallint
Indicator_Current_health_expenditure_CHE_as_percentage_of_gross_domestic_product_GDP	float

Table: insurance

Columns:

Entity	nvarchar(50)
Code	nvarchar(50)
Year	smallint
Indicator_Current_health_expenditure_CHE_as_percentage_of_gross_domestic_product_GDP	float

Table: no_ins

Columns:

Entity	nvarchar(50)
Code	nvarchar(50)
Year	smallint
Percentage_of_persons_without_health_insurance	float

Table:share_gdp

Columns:

Entity	nvarchar(50)
Code	nvarchar(50)
Year	smallint
Domestic_general_government_health_expenditure_of_GDP	float

Data transformation and cleaning:

Created a Hadoop cluster using AWS Elastic MapReduce(EMR) which runs on 2 m5.xlarge EC2 instances one is primary instance and one is core instance

The screenshot displays the AWS Management Console interface for an Amazon EMR cluster. The top navigation bar shows the AWS logo, a search bar, and the user's profile. The main content area is titled "My cluster" and includes a summary section with four columns: Cluster info, Applications, Cluster management, and Status and time. The cluster ID is j-2A5H6N96VNGSN, and its status is "Waiting". Below the summary, there are tabs for Properties, Bootstrap actions, Instances (Hardware), Steps, Applications, Configurations, Monitoring, Events, and Tags (1). The Properties tab is active, showing the operating system (Amazon Linux release 2.0.20231218.0), cluster logs, and cluster termination options. The cluster logs section shows that archive log files to Amazon S3 are turned on, and encryption for logs is turned off. The cluster termination section shows that the termination option is "Manually terminate cluster" and termination protection is turned on.

Configured the cluster in Putty terminal and installed virtual environment for airflow which will be used for workflow scheduling

```
[hadoop@ip-172-31-26-236 ~]$ sudo python3 -m pip install virtualenv
```

Data Extraction:

Used Spark with scala to import the datasets stored in S3 bucket and named it as task_t1

Creating a spark session

```
import org.apache.spark.sql.SparkSession

object task_t1 {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName("task_t1")
      .getOrCreate()

    val s3BasePath = "s3n://bunny970/Datasets/"
```

Creating a list from the dataset names and used foreach loop to import the datasets present in the bucket to store it in HDFS

```
val datasets = List("annual-healthcare-expenditure-per-capita.csv", "expenditure-of-the-national-health-service-nhs-in-the-uk.csv",
"health-insurance-coverage-in-the-us.csv", "health-protection-coverage.csv", "percentage-of-persons-without-health-insurance-coverage-us.csv",
"public-health-expenditure-share-GDP-OWID.csv", "public-health-insurance-coverage.csv",
"public-healthcare-spending-share-gdp.csv", "share-of-out-of-pocket-expenditure-on-healthcare.csv", "share-of-public-expenditure-on-healthcare-by-
total-healthcare-expenditure-gdp.csv")

datasets.foreach(dataset => {
  val s3Path = s"$s3BasePath/$dataset"
  val data = spark.read.csv(s3Path)
  val outputPath = s"/home/hadoop/health_datasets/$dataset"
})

spark.stop()
}
```

Data transformation:

Created a spark job with object task_t2 for data cleaning and transformation

```
import org.apache.spark.sql.{SparkSession, DataFrame}

object task_t2 {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName("task_t2")
      .getOrCreate()

    val datasets = List(
      "annual-healthcare-expenditure-per-capita.csv",
      "expenditure-of-the-national-health-service-nhs-in-the-uk.csv",
      "health-insurance-coverage-in-the-us.csv",
      "health-protection-coverage.csv",
      "percentage-of-persons-without-health-insurance-coverage-us.csv",
      "public-health-expenditure-share-GDP-OWID.csv",
      "public-health-insurance-coverage.csv",
      "public-healthcare-spending-share-gdp.csv",
      "share-of-out-of-pocket-expenditure-on-healthcare.csv",
      "share-of-public-expenditure-on-healthcare-by-country.csv",
      "total-healthcare-expenditure-gdp.csv"
    )
  }
}
```

There are no Null values present in the dataset and no major transformations required the only transformations required for the datasets is removing irrelevant values present in the entity column, in entity column there are health and insurance details are present based on country and country code and the region but there are some values like High income, Middle income, and Low income which have NULL values and contain invalid year so removed the records with these values.

Created a directory in HDFS to store the transformed datasets.

```
datasets.foreach(dataset => {
  val inputPath = s"/home/hadoop/health_datasets/$dataset"

  val data = spark.read.option("header", "true").csv(inputPath)
  val entitiesToRemove = List(
    "High income",
    "Low and middle income",
    "Low income",
    "Lower middle income",
    "Middle income",
    "Upper middle income"
  )

  val filteredData = data.filter(col("Entity").isin(entitiesToRemove: _*))
  val outputPath = s"/home/hadoop/health_datasets_updated/$dataset"
  filteredData.write.option("header", "true").csv(outputPath)
})

spark.stop()
}
```

Data Export:

Created a spark job to export the transformed datasets to S3 bucket

```
import org.apache.spark.sql.{SparkSession, DataFrame}

@object task_t3 {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder
      .appName("task_t3")
      .getOrCreate()

    val transformedDatasets = List(
      "annual-healthcare-expenditure-per-capita.csv",
      "expenditure-of-the-national-health-service-nhs-in-the-uk.csv",
      "health-insurance-coverage-in-the-us.csv",
      "health-protection-coverage.csv",
      "percentage-of-persons-without-health-insurance-coverage-us.csv",
      "public-health-expenditure-share-GDP-OWID.csv",
      "public-health-insurance-coverage.csv",
      "public-healthcare-spending-share-gdp.csv",
      "share-of-out-of-pocket-expenditure-on-healthcare.csv",
      "share-of-public-expenditure-on-healthcare-by-country.csv",
      "total-healthcare-expenditure-gdp.csv"
    )

    transformedDatasets.foreach(transformedDataset => {
      val inputPath = s"/home/hadoop/health_datasets_updated/$transformedDataset"

      val transformedData = spark.read.csv(inputPath)

      val s3OutputPath = s"s3n://bunny970/Datasets_updated//$transformedDataset"

      transformedData.write.csv(s3OutputPath)
    })

    spark.stop()
  }
}
```

Built JAR files for task_t1, task_t2 and task_t3 objects for using the jar files as spark jobs for workflow scheduling using apache-airflow and uploaded it in S3 bucket

Objects

Properties

Objects (3) Info

🔄

📄 Copy S3 URI

📄 Copy URL

📄 Download

🔗 Open

🗑 Delete

⌵ Actions

Create folder

📤 Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍 Find objects by prefix

< 1 > 🔄

☐	Name	Type	Last modified	Size	Storage class
☐	 task_t1.jar	jar	January 16, 2024, 20:10:25 (UTC+05:30)	2.8 KB	Standard
☐	 task_t2.jar	jar	January 17, 2024, 10:23:25 (UTC+05:30)	3.2 KB	Standard
☐	 task_t3.jar	jar	January 17, 2024, 10:30:30 (UTC+05:30)	2.9 KB	Standard

Importing the jar files and submitting the spark job to ensure they are working correctly or not

```
airflow-venv health_datasets
[hadoop@ip-172-31-26-236 ~]$ aws s3 cp s3://bunny970/jar\ files/task_t1.jar .
download: s3://bunny970/jar files/task_t1.jar to ./task_t1.jar
[hadoop@ip-172-31-26-236 ~]$ ls
airflow-venv health_datasets task_t1.jar
[hadoop@ip-172-31-26-236 ~]$ spark-submit --class task_t1 task_t1.jar
24/01/16 14:41:21 INFO SparkContext: Running Spark version 3.4.1-amzn-2
24/01/16 14:41:21 INFO ResourceUtils: =====
24/01/16 14:41:21 INFO ResourceUtils: No custom resources configured for spark.driver.
24/01/16 14:41:21 INFO ResourceUtils: =====
```

Workflow Automation:

Used apache airflow to schedule the tasks by using the 3 JAR files
Created a DAG named task_dag.py
Setting up default parameters which define the behaviour of the DAG

```
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}
```

Creating the airflow DAG

```
dag_combined = DAG(
    'task_dag',
    default_args=default_args,
    description='Combined DAG for Spark Jobs and EMR Steps',
    schedule_interval='@daily',
)
```


Configuring `emr_step_adder` and `emr_step_checker` using Apache Airflow. The `emr_spark_step` dictionary defines a Spark step with specific configurations, and the `EmrAddStepsOperator` adds this step to an EMR cluster specified by `'job_flow_id'`. The subsequent `EmrStepSensor` monitors the execution of the Spark step by referencing the step ID obtained from the output of the previous operator, ensuring the DAG progresses based on the success or failure of the EMR Spark job.

```
emr_spark_step = {
    'Name': 'EMRSparkStep',
    'ActionOnFailure': 'CONTINUE',
    'HadoopJarStep': {
        'Jar': 'command-runner.jar',
        'Args': ['spark-submit', '/path/to/your/app_s3_to_ec2.jar']
    }
}

emr_step_adder = EmrAddStepsOperator(
    task_id='add_emr_spark_step',
    job_flow_id='j-2A5H6N96VNGSN',
    aws_conn_id='aws_default',
    steps=[emr_spark_step],
    dag=dag_combined
)

emr_step_checker = EmrStepSensor(
    task_id='watch_emr_spark_step',
    job_flow_id='j-2A5H6N96VNGSN',
    step_id="{{ task_instance.xcom_pull(task_ids='add_emr_spark_step')['StepIds'][0] }}",
    aws_conn_id='aws_default',
    dag=dag_combined
)
```

Created 3 tasks `t1`, `t2`, `t3` using Bash operators which represents 3 spark jobs with `spark-submit` for the jar files

```
t1 = BashOperator(
    task_id='t1_s3_to_ec2',
    bash_command='spark-submit /home/hadoop/task_t1.jar',
    dag=dag_combined,
)

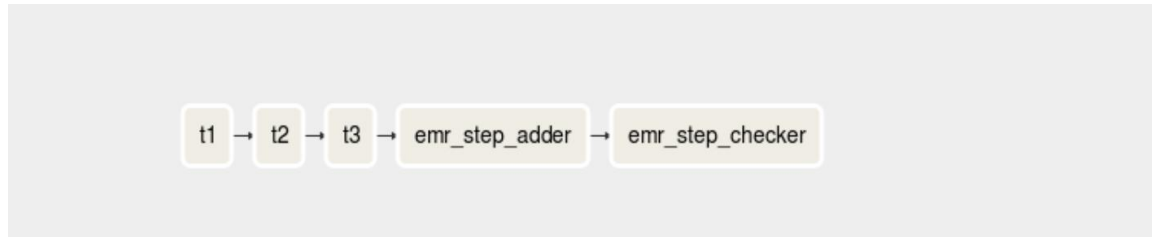
t2 = BashOperator(
    task_id='t2_transform_and_save',
    bash_command='spark-submit /home/hadoop/task_t2.jar',
    dag=dag_combined,
)

t3 = BashOperator(
    task_id='t3_export_to_s3',
    bash_command='spark-submit /home/hadoop/task_t3.jar',
    dag=dag_combined,
)
```

Setting up dependencies where each tasks execute subsequently where task t1 executes first followed by t2 and t3 then emr_step_adder and emr_step_checker they depend on successful execution

```
t1 >> t2 >> t3 >> emr_step_adder >> emr_step_checker
```

Graph view of the DAG



Data Analysis

Loaded the transformed datasets to AWS Athena and used Hive query language for data retrieval

Table creation:

```
CREATE EXTERNAL TABLE IF NOT EXISTS `health_care`.`health_gdp` (  
  `country` string,  
  `code` string,  
  `year` int,  
  `exp` double  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe'  
STORED AS INPUTFORMAT 'org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat'  
  OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat'  
LOCATION 's3://bunny970/Datasets_updated/'  
TBLPROPERTIES ('classification' = 'parquet');
```

Query to get average healthcare expenditure as a percentage of GDP for each country

Query 1 ⋮	
1	SELECT country, AVG(exp) AS avg_health_exp
2	FROM health_gdp
3	GROUP BY country;

Query to find the year with highest overall healthcare expenditure across all countries

Query 1 ⋮	
1	SELECT year, SUM(exp) AS total_health_exp
2	FROM health_gdp
3	GROUP BY year
4	ORDER BY total_health_exp DESC
5	LIMIT 1;

Query to get countries with insurance coverage less than 20%

Query 1 ⋮	
1	SELECT country, year, cov
2	FROM insurance
3	WHERE cov < 20 AND cov IS NOT NULL;

Query to get countries with decrease in healthcare insurance coverage from 1910 to 1975

Query 1 ⋮	
1	SELECT country, cov_1910, cov_1975, cov_1975 - cov_1910 AS coverage_change
2	FROM (
3	SELECT country,
4	MAX(CASE WHEN year = 1910 THEN cov END) AS cov_1910,
5	MAX(CASE WHEN year = 1975 THEN cov END) AS cov_1975
6	FROM insurance
7	GROUP BY country
8	) subquery
9	WHERE cov_1975 < cov_1910;

Query to get top 3 regions with lowest healthcare expenditure

Query 1 ⋮	
1	SELECT Entity AS region, year, exp
2	FROM health_exp
3	WHERE code IS NULL
4	ORDER BY exp ASC
5	LIMIT 3;

Query to calculate yearly percentage change in health expenditure for each region

Query 1 ⋮	
1	SELECT Entity AS region, year, exp,
2	(exp - LAG(exp, 1) OVER (PARTITION BY Entity ORDER BY year)) / LAG(exp, 1) OVER (PARTITION BY Entity ORDER BY year) * 100 AS percentage_change
3	FROM health_exp
4	WHERE code IS NULL
5	ORDER BY Entity, year;

Query to find the years with lowest and highest percentage of people without health insurance

Query 1 :

1

2

3

4

5

6

7

8

9

SELECT year, without_ins

FROM no_ins

ORDER BY without_ins DESC

LIMIT 1

UNION

SELECT year, without_ins

FROM no_ins

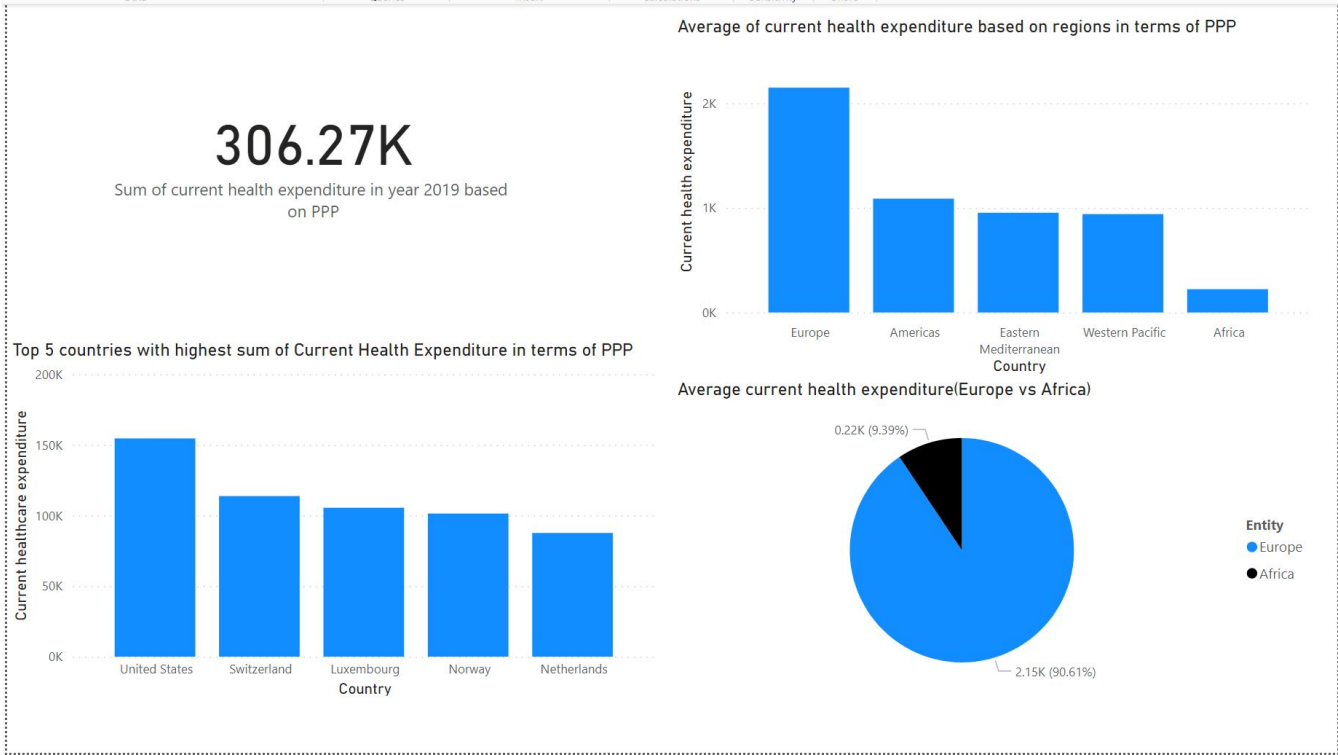
ORDER BY without_ins ASC

LIMIT 1;

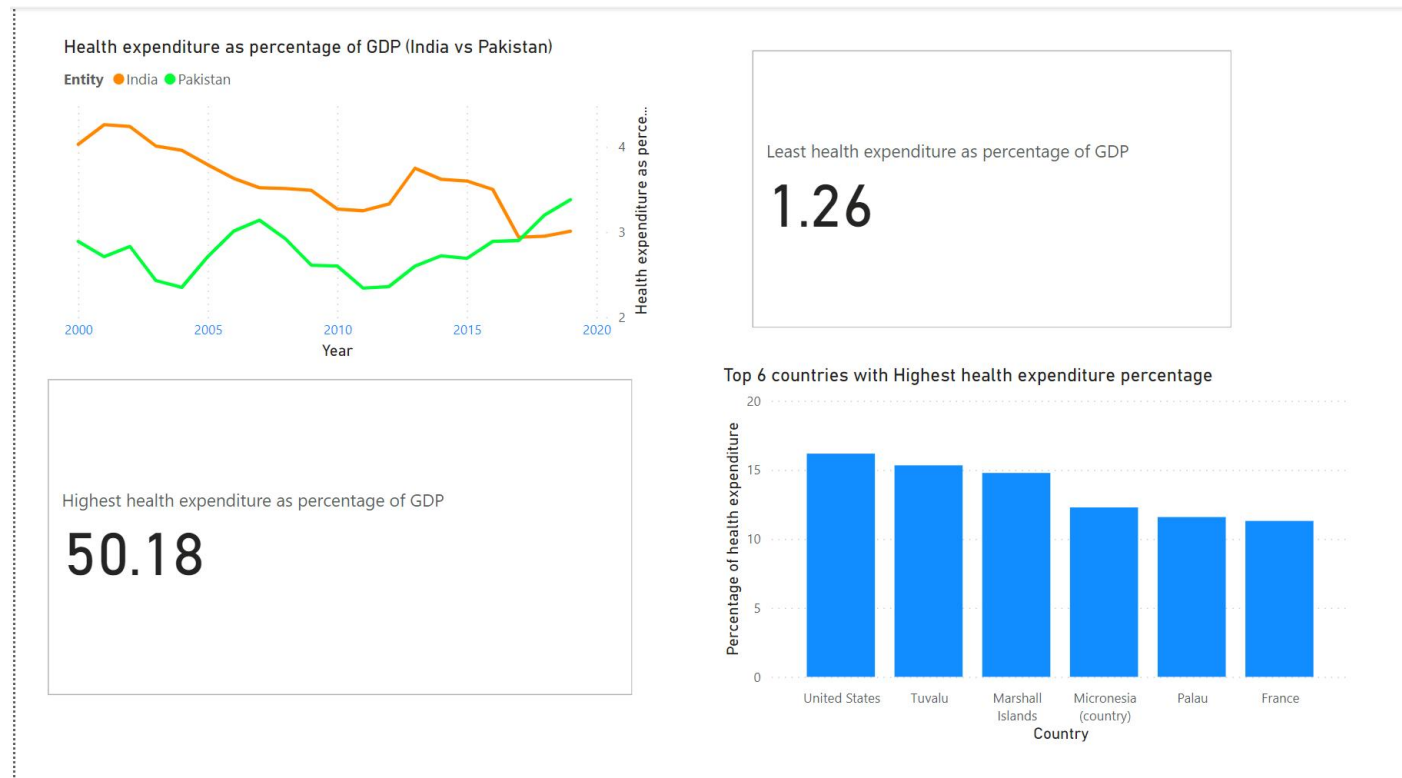
Data visualization:

Used power BI for data visualization

Visual on current health expenditure based on countries and regions in terms of purchasing power parity

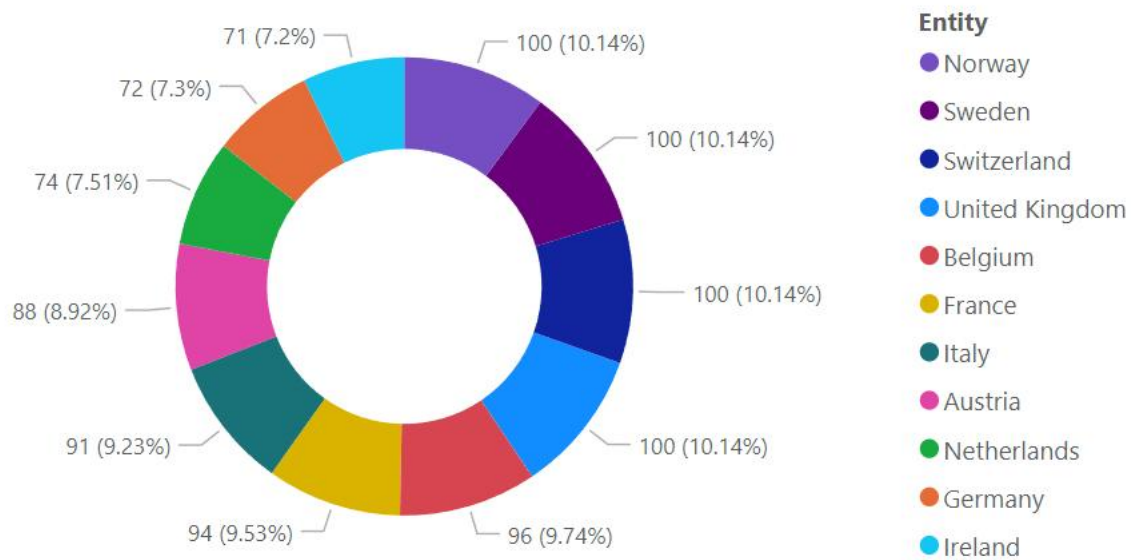


Visual on health expenditure as percentage of GDP based on countries

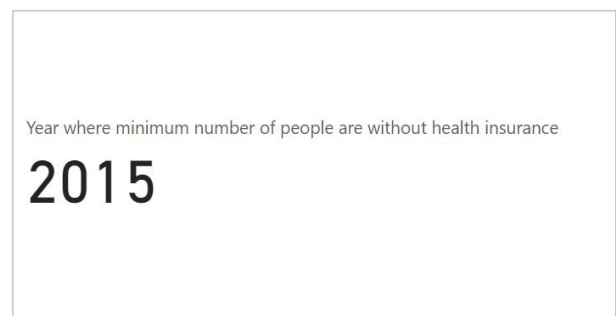
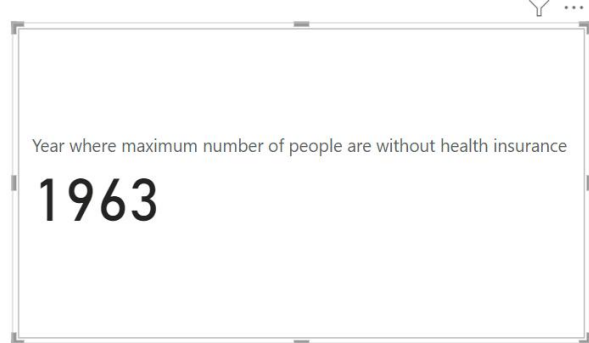


Visual on health insurance coverage by every country in europe in 1975

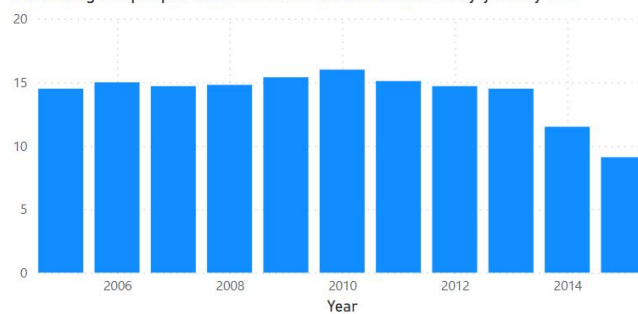
Health insurance coverage by country in year 1975



Visual on percentage of people without health insurance in USA



Percentage of people without health insurance in USA by year by Year



Conclusion:

In summary, the healthcare project used Spark and Scala to handle data, stored information on AWS EMR cluster and S3, scheduled tasks with Apache Airflow, retrieved data quickly with Athena, and performed insightful analysis using Power BI.